

Nested Blockchain Transaction for Multiple Methods of a Smart Contract

Nested Transactions Supporting Multiplicity of Smart Contract Methods

Chris, G., Liu *

Dalhousie University, Chris.Liu@dal.ca

Peter, Bodorik

Dalhousie University, Peter.Bodorik@dal.ca

Dawn, Jutla

Dawn.Jutla@gmail.com

Research on blockchains addresses multiple issues, with one being the development of smart contracts. In developing smart contracts, blockchains have complex constraints for applications, such as maintenance of smart contracts that is challenged by the fact that smart contracts are stored on a blockchain that is immutable. Blockchain applications that involve collaborations of multiplicity of actors, such as in trade of goods and services, give rise to additional complexity in that transaction activities tend to be long term, perhaps hours or days, and thus spanning multiplicity of methods of a smart contract. The problem is that a blockchain only supports the concept of a transaction within the scope of an execution of a single smart contract method. This is further complicated by privacy requirements of actors who participate in a transaction. We compare the properties of long-term transactions in Databases (DB) and blockchain environments and discuss their mismatch. We then propose how to represent long-term blockchain transactions in a smart contract and propose properties of such transactions that should be supported by a long-term blockchain transaction mechanism. Furthermore, we propose alternative mechanisms to support such transactions and their properties. We evaluate the proposed alternative mechanisms in terms of the support for the required and desirable transactional properties and their cost of execution. Finally, we describe how our approach is used to support nested blockchain transactions.

CCS CONCEPTS • Computing methodologies → Distributed computing methodologies • Networks → Network services • Computer systems organization → Architectures • Software and its engineering → Software organization and properties • Information systems → Information systems applications • Computing methodologies → Modeling and simulation

Additional Keywords and Phrases: Blockchain, Trade of Goods and Services, Transactions, Long-term Transaction, Nested Transactions, Transactions for Multiplicity of Smart Contract Methods, Sidechain, Privacy for Blockchains, Optimistic Methods

* Place the footnote text for the author (if applicable) here.

1 INTRODUCTION

In comparison to the early days of research on distributed ledgers and cryptocurrencies, such as digital cash [Chaum 1983], the prominence and popularity of the distributed technology has risen dramatically since the introduction of the Bitcoin cryptocurrency [Nakamoto 2008]. The start of public trading of Bitcoin cryptocurrency in 2010 and its success [Marr 2017] resulted in many blockchain-based platforms, with many great successes but also many failures. The concept of a smart contract [Szabo 1997, 2018] was adopted by Ethereum [Buterin 2015] that also included a smart contract expressed as a collection of methods written in a Turing-complete high-level language, which is also stored on the ledger. Although storing the smart contract on the blockchain ledger provides the smart contract code security of a blockchain ledger, it does increase challenges in designing and developing smart contracts due to the constraints arising from the use of the blockchain technology and its architecture.

Blockchain augments the distributed ledger technology, which preceded the blockchains, with smart contracts that are stored on the blockchain. That is, a blockchain is a replicated ledger that stores, in addition to data, also smart contracts that are programs used to manipulate the ledger data. In addition, the blockchain infrastructure utilizes cryptographic concepts and methods to provide the blockchain's desirable properties of trust, immutability, availability, and transparency, amongst others. However, as a new technology, blockchain and their smart contracts pose new challenges in both the blockchain infrastructure and in developing smart contracts and applications that use them.

Blockchain scalability, transaction throughput, and high costs are some of the main issues investigated by research on the blockchain infrastructure. The high cost of the Proof of Work consensus algorithm [Proof of work, 2023], which is used in some prominent blockchains to ensure that a block of data to be appended to the chain is valid in face of errors or attacks, has been an impediment to adoption of blockchains. Consequently, the Ethereum public blockchain adopted the network layer-2 approach to replace its proof-of-work consensus algorithm with the proof-of-stake mechanism in order to increase the Ethereum's throughput. In essence, another blockchain layer/network has been added to record proofs of transactions performed by the first layer.

Smart contracts and their development and maintenance received much attention by the research and development communities as can be seen by literature surveys in [Taylor 2019, Khan 2021, Vacca 2021, Belchior 2021, Saito 2016, Garcia-Garcia 2020, Lauster 2020, Levasseur 2021]. Khan et al. (2021) survey and classify research on blockchain smart contracts into two broad categories of improvement and usage, in which the improvement category includes research and tools to improve smart contracts either when writing them or once they exist. Improvement approaches for writing of smart contracts generally rely on defining a programming language that facilitates creation of smart contract with desirable properties. However, although these approaches lead to smart contract programs with formally proven safety properties, they have not been adopted in practice as they exhibit, or are perceived to exhibit by software developers, complexity that deters their adoption by developers who ultimately decide how well-accepted a programming language is. To alleviate such issues, some researchers use a well-known and used Business Process Management Notation (BPMN) to model the application requirements and transform a BPMN model automatically into the methods of a smart contract(s) [Weber 2016, López-Pintado 2019, Tran 2018, López-Pintado 2019, Bodorik 2022, Liu 2021]. In some instances, the BPMN model is augmented with additional information that is required for the transformation process from a BPMN model

into methods of a smart contract. The additional information deals primarily with details on information/data as it flows through the PBMN model.

In addition to already described issues in developing smart contracts, there is an additional problem arising in blockchains supporting a large class of applications that involve long-term transactions involving activities of several collaborators cooperating in a distributed application that can be conveniently represented as a collection of smart contract methods and that would be awkward and hard to represent as one smart contract method. The problem is that a blockchain supports only a transaction that contains ledger updates made by an execution of any one of the methods of the smart contract. How to support a blockchain transaction that spans execution of a multiplicity of executions of methods of a smart contract is the main topic of this paper. The following sub-sections provide the motivation, objectives, our contribution to solving this problem, and an outline of further sections.

1.1 Motivation

Trade of goods and services, as opposed to securities, includes collaborative activities that arise with a sale or trade of goods and/or services between two or more parties. Furthermore, for higher priced sale items, such as combines or other special machinery, typically banks are involved in providing financial instruments supporting trade, such as a letter of credit that, in addition to the buyer and seller parties, also involves the buyer's and seller's banks and other parties. The buyer's bank will guarantee payment of goods to the seller's bank upon receiving the goods or services and other parties may be involved in the trade, such as transporters/shippers for the delivery of the purchased goods, insurance companies for risk mitigation, and regulators if the goods are crossing international or state/provincial/municipal boundaries. Thus, trade transactions involve many parties that collaborate in activities that may be long term, e.g., spanning, minutes, hours, or days, as the collaborative activities may involve different software or human events that trigger invocation of the smart contract methods by a distributed application. We shall refer to such transactions simply as *long-term collaborative transactions*, or as *long-term transactions*.

The main consequence of a long-term transaction in a smart contract is that it involves invocation of a multiplicity of smart contract methods as different actors may need to provide different information to a long-term transaction at different times or when certain events arise - it is not easy to support such a transaction in blockchains due to the mismatch between the long-term transactions requirements, involving execution of a number of methods of a smart contract, and the definition of a blockchain transaction that has a scope of an execution of any individual smart contract method that updates the ledger. The main consequence of a long-term transaction in a smart contract is that it involves invocation of more than one, and potentially many, methods of a smart contract, and that is something that a blockchain does not support: A transaction in a blockchain is the result of the ledger updates of an execution of a smart contract method – currently in a blockchain, a transaction cannot include results of executing more than one smart contract methods and that is what a long-term transaction requires. There are two main issues in supporting long-term transactions:

- a) The first issue is how to specify a long-term transaction within the context of a smart contract and which properties of a long-term transaction should be supported by a blockchain mechanism.
- b) The second issues if of how to provide a mechanism to support long-term transaction and its properties.

Furthermore, a transaction may involve sub-transactions. For instance, a purchase using a letter of credit is likely to involve sub-transactions, such as for shipping, insurance, or customs. Thus, sub-transactions also need to be supported.

1.2 Objectives

Our objectives include defining the concept of a long-term blockchain transaction. Although thus far we described a long-term transaction in general terms, the properties of such transactions that need to be supported by the transaction mechanism also need to be identified. Thus, our objectives also include finding a mechanism to support the long-term transactions on blockchains, which is a mechanism that provides the support for the long-term transactions. More specifically, our objectives are:

- Review transaction and their properties in DBs and in Blockchains.
- Propose how to define, or represent, a long-term transaction within the context of blockchains.
- Propose a mechanism(s) to support a long-term blockchain transaction and its properties.
- Evaluate the proposed mechanism(s).

1.3 Contributions

- Long-term transaction concept for blockchains: Motivation is provided for support of a long-term transaction in a blockchain that spans execution of a multiplicity of methods of a smart contract. DB and blockchain transactions are analyzed to determine that the properties of Atomicity, Consistency, Isolation, and Durability (**ACID**) properties defined for DB systems also need to be supported while access control and privacy should be supported by a mechanism to support long-term transactions on a blockchain.
- Long-term Blockchain (**LtB**) transaction representation: We propose how to represent/define an LtB transaction in current blockchains and which transactional properties should be supported.
- Automation of long-term transaction mechanism: The proposed LtB transaction mechanism reduces the developer's involvement in facilitating the LtB transaction mechanism: The developer simply creates the smart contract that contains the LtB transaction definition and then invokes a tool that amends the smart contract methods with appropriate patterns that implement the support of LtB transactions and their properties.
- LtB transaction mechanisms and their evaluation: We propose alternative LtB transaction mechanisms to support LtB transactions, wherein the alternative mechanisms are based on different architectures leading to varying support for the privacy property. We evaluate the different mechanisms and show that they support the ACID and access control properties. We compare the different alternative mechanisms in terms of their relative levels of support for privacy and we also provide rough estimates for the execution cost of the alternative mechanisms.
- Nested LtB transactions: We extend the proposed mechanism to support nested LtB transactions, wherein a transaction may contain independent sub-transactions.

1.4 Outline

The second section reviews and compares the concepts of a transaction in the context of DBs and in the context of blockchains. It compares the two concepts, including their transactional properties, to propose

how to represent an LtB transaction in current blockchains and which of the transactional properties must be or should be supported.

The third section outlines several alternative approaches/mechanisms that can be used to support the LtB transactions, while the fourth section evaluates those alternative mechanisms in terms of their support of the LtB transaction properties and in terms of overhead costs.

Of course, like DBs, the concept of a transaction is insufficient as, particularly in trade of goods and services wherein transactions may have sub-transactions and thus leading to nested transaction. Thus, in the fifth section we extend the previously described mechanism with support for nested transactions. The penultimate section provides related work, while the final section provides summary and concluding remarks.

2 LONG-TERM BLOCKCHAIN TRANSACTIONS

We overview the concept of transactions in the context of DBs and blockchains and then compare them in terms of their properties to discover their similarities and differences in order to determine which of the properties must be supported or should be supported by the long-term blockchain transaction mechanism and how to represent an LtB transaction. We start with an overview of DB transactions, because DBs and their transactions are more mature than those of blockchain and were developed to support data needs of various applications, including applications supporting collaborative activities. We then review the blockchain transactions and their properties and compare them to those of DB transactions. The comparison will be used to determine which of the transactional properties must be supported and which properties should be supported by the LtB transaction mechanisms, and how to represent a long-term transaction in a smart contract that utilizes blockchain infrastructure with the current definition of a blockchain transaction.

However, before we proceed, we review the usage of the term “transaction” that, of course, depends on the context, such as a transaction in DBs or in blockchains:

- *Transaction* ... used as a general term or in a specific context that implies the transaction type.
- *DB transaction* ... refers to a transaction (and its properties) as currently used for DB systems.
- *Blockchain transaction* ... refers to a transaction and its properties as currently defined within the context of a smart contract on a blockchain.
- *Long-term transaction* ... refers to a transaction that involves collaboration of multiple actors that takes a long time and in which collaborator's activities are performed at unpredictable times within a transaction as they need the collaborators' inputs.
- *Long-term Blockchain (LtB) transaction* ... refers to a transaction that spans multiple methods of a smart contract – one of the *objectives* of this paper. We refer to such a transaction as an *LtB transaction* for short.
- *LtB transaction mechanism* ... refers to a mechanism to support a long-term transaction within the context of a blockchain – another *objective* of this paper.
- *Actor* ... software that is being executed on behalf and under the control of a user. Actor's delay may be slow if the user's (human) response is required, or it may be fast if the response is provided by software actor executing on user's behalf.
- *User* ... we use the terms Actor and User interchangeably.

- *Participant* ... Actor or User that participates in a smart contract or that participates in a long-term transaction that is a part of a smart contract.

2.1 DB Transactions and Their Properties

Our review of DB transactions will be brief, as the concept of DB transactions is well established and is being taught in DB courses at any college/university level.

Definition: A DB transaction is a sequence of reads and writes issued against a DB, wherein the reads and writes are delineated explicitly by the transaction begin and end operations/instructions.

2.1.1 DB ACID Properties

The DB system ensures that the transaction operations are such that the properties of *atomicity* (all or nothing), *consistency* (correctness when executing transactions concurrently), *isolation* (no dirty reads), and *durability* are satisfied. These properties are referred to briefly as DB transactional or **ACID** (*Atomicity, Consistency, Isolation, Durability*) properties.

2.1.2 DB Access Control and Privacy

In a relational DB, access control is performed on access to a relation in that a user, issuing a **Create/Read/Update/Delete (CRUD)** operation on a DB, must have appropriate privileges to issue that operation. To access a relation, the user provides her credentials, and a DB access control mechanism determines whether the user has access to the relation for the operation requested by the user. Control of which users may issue which of the CRUD operation on a relation is controlled by the relation's owner (or the DB administrator who has all privileges).

Moreover, access control to the relations is also used to ensure privacy – to view data in a relation, access control to the relation ensures that the user has appropriate credentials to view the data stored in a relation.

2.2 Blockchain Transactions and Properties

Before we proceed with the discussion on blockchain transactions, we shall briefly review the developer's view of reading and writing to the blockchain ledger. The blockchain ledger is immutable, which means that only the ledger reads and appends to the end of the ledger are permitted. As writing methods of a smart contract using only reads and appends would be challenging and error prone, to ease the developer's task to write smart contracts, the blockchain infrastructure provides to the developer a view of the ledger data as an associative data store, such as the Level DB supported by both Ethereum and Hyperledger, or the Couch DB supported by the Hyperledger Fabric. The associative DB provides the usual associative data-store operations to write and read values:

- *Write (key, value)* ... The method takes two inputs, one being the key, and the second one containing a value that is stored on the ledger using that key.
- *Value = Read (key)* ... To read from the ledger, the user provides a single input parameter, the object key that is used to retrieve from the ledger (i.e., read the ledger) the value associated with the key.

Although the associative data store makes no assumption on the structure of the value that is read or written, we shall assume that the value that is written or read is in a form of an object.

Each time a smart-contract method writes to the ledger using a key k , the blockchain infrastructure creates a new instance/version for that object if it exists already. Furthermore, the blockchain infrastructure stores the new object instance on the ledger in a Merkle-tree data structure, and it keeps track of the order of the versions of any object. Thus, a developer views the ledger as an associative data store, while the blockchain infrastructure performs the transformation of the reads and writes on the associative data store to the reads and appends of new object versions on the immutable ledger itself.

In the following text, when we refer to the ledger *reads* and *writes*, we refer to the developer's view of the ledger data using the associative data store, while keeping in mind that the blockchain infrastructure performs the transformation of developer's operations to the operations on the immutable ledger itself. Hence, when we refer to a ledger *write*, we are referring to the ledger's *append* operation.

Documentation on the various blockchains generally states that a transaction is any change to the state of the ledger. Thus, we define a blockchain transaction accordingly:

Definition: A blockchain transaction is the sequence of reads and writes made on the ledger by an execution of any one of the smart contract methods.

Thus, any execution of a smart contract method that writes to the ledger results in a blockchain transaction and, consequently, the scope of a transaction is an execution of any smart contract method. We already discussed the DB transactional properties, that is the ACID properties that all major relational DB systems support. We now briefly discuss if and how these properties are supported in blockchains, after which we shall discuss additional transactional properties that give rise to differences between the DB and blockchain transactions.

2.2.1 Blockchain Transaction ACID Properties

In a blockchain, updates made by an execution of a smart contract method are submitted for inclusion in a block that is appended to the blockchain. Block validation ensures that updates made by a smart contract method are consistent relative to the state of the ledger and relative to all transactions within that block. Consequently, writes of a smart contract method form an *atomic transaction* in that either all writes made by a smart contract method are performed on the ledger or none are. Furthermore, the consensus method ensures *consistency*, because all ledger updates included in a block are validated to be consistent relative to the ledger and relative to the other transactions within the same block. Furthermore, as smart contract methods may access the transaction's updates only after the block is appended to the blockchain, dirty reads are prevented and hence *isolation* is also achieved. Finally, as updates are written to the ledger, which is replicated, successfully completed/committed transactions are *durable* in that they are not lost after the transaction successfully completes.

2.2.2 Blockchain Access Control and Privacy

Blockchains are classified generally into two categories, *permissioned* and *public* blockchains. Public blockchains, as the name indicates, have no access control as anyone can get an account and execute any methods of any smart contract on a public blockchain. Permissioned blockchain force a user to have an account on a blockchain, wherein the account is used for ensuring *access control on a per smart*

contract basis. That is, a user must have an account with appropriate privileges that permit the user to access the ledger and to access a smart contract(s). If a user has the privilege to access a smart contract, that user may use any of the methods of that smart contract.

Another issue that is raised when discussing the properties of blockchain operations is that of *privacy*. The privacy issue arises in blockchains because any smart contract method has access to all data stored on the ledger. And privacy is very important for certain types of applications in trade of goods and services, such as in supply chain applications.

Organizations participating in supply-chain activities, and in many other applications that involve trade of goods and services, are at the same type partners and competitors. Thus, for instance, in supply chains supported by smart contracts, two organizations participating in a supply chain need to share information about the activities between them, information that is stored on the ledger. However, other organization, who may be competitors, also may participate in the same supply chain, and that is a problem as any organization participating in the supply chain has access to all information stored on the ledger and hence all information stored in the supply chain – and for most supply chains this is not acceptable. Exception to this do exist when a blockchain supply chain is governed by a big corporation or a regulatory body, wherein the governance requires that if suppliers wish to participate in the supply chain, they must provide the required information even if it is visible to all suppliers participating in the supply chain.

To support privacy, Hyperledger fabric provides for *channels* that can be used by any subset of organizations to store information on that channel, which, for all practical purposes, is a separate blockchain. For further privacy requirements, Hyperledger also provides for *private data* that is accessible only to designated actors.

However, privacy issues arise also within the scope of a smart contract if the contract contains an LtB transaction. Actors who participate in the smart contract, but are not participants to an LtB transaction, in general, should not have access to the data written as a part of an LtB transaction. And this also applies to each transaction if a smart contract has more than one. A blockchain application may have several transactions, one for negotiating the price, one for transport of goods to the port of departure, one for activities required due to international trade (duties, permits, etc.), and yet another for accepting the goods. Clearly, the transaction for the process of price negotiation is independent of the other transactions, and the users participating in one transaction should not have access to the particulars of another transaction if they do not participate in it. In general, participants in transactions dealing with trade of goods and services may be both partners and competitors – hence privacy is important in such situations.

2.3 Properties of LtB Transactions

As mentioned earlier, although the properties of long-term transactions are not formally defined in literature, those properties need to be defined so that enforcement of those properties can be supported by the mechanism supporting long-term transactions. To identify which properties should an LtB transaction mechanism support, we review the DB and blockchain transactional properties to determine which properties must or should be supported by the LtB transaction mechanisms:

- a. For any transactional property that is supported by both the DB and blockchain systems, we shall consider it as a required property that must be enforced.

- b. For any property supported by one of the DB or blockchain systems, it will be deemed as a desirable property that should be provided.

To summarize from the previous sections, both DB and blockchain transactions do support the ACID properties, but they differ in operations and the scope of a transaction relative to the application's access to the DB/blockchain: In DBs, a transaction may include many CRUD operations that are executed against the DB, wherein an application may invoke arbitrary operations in arbitrary sequence.

Clearly, as both the DB and blockchain systems support the ACID properties, the *ACID properties must be supported for LtB transactions* also. In DBs, a transaction may consist of a sequence of CRUD operations issued against the DB, wherein each CRUD operation may be submitted to the DB as an individual operation issued independently by the application. In a blockchain, by contrast, a transaction is the set of ledger reads and writes as executed by a method of a smart contract. Thus, to support the concept of an LtB transaction, the limitation imposed by the blockchain infrastructure, that a transaction is limited to the ledger updates made by execution of any one smart contract method, must be overcome to support *LtB transactions that involve ledger writes made by a multiplicity of invocations of the smart contract methods*.

In addition to the ACID properties, although both DB and blockchain systems provide for access control, the concepts are quite different. In relational DBs, access control is based on controlling access to relations and different access permissions may be specified for the different CRUD properties as permitted for each user. Blockchains also provide for access control, but not to the ledger data, but rather to the methods of a smart contract – but only for the permissioned blockchains. In a permissioned blockchain, blockchain provides access control for accessing the methods of a blockchain smart contract based on the identity of the actor who causes the invocations of a smart contract method.

In a public blockchain, anyone can create an account and write smart contract that reads and writes the ledger data. Access control to the methods of the smart contract is not provided by the public blockchain infrastructure. However, transfer of funds stored in in a contract is controlled: Only the smart contract owner, who has the private key associated with the user account has the right to transfer or “spend” funds associated with the smart contract.

From the above we surmise that *access control on who can access methods of an LtB transactions* is desirable and should be supported.

As was previously discussed, privacy is viewed as the ability to ensure that only certain users may view/access an object written to the ledger. In DBs, privacy is provided by controlling access to relations: A user can view data in a relation only if the user has read access to that relation. In blockchains, however, there is no restrictions on read/write access to the ledger - if a user can create smart contracts and execute its methods, there is no restriction on which ledger data can be read by such methods. However, an LtB mechanism should support privacy by allowing *only participants of the LtB transaction to view ledger data written by an LtB transaction*.

In summary, to support the long-term blockchain transactions:

1. An LtB transaction must be such that it includes ledger reads and writes as performed by a specified subset of the methods of a smart contract.
2. The ACID properties must be supported.

3. Access control, on which users can access methods of a LtB transaction, should be supported.
4. Privacy should be supported to ensure that only the actors who can access the transaction operations can view the results of operations within that transaction, i.e., that they should not be viewed by other actors who are not participating in the LtB transaction.

2.4 LtB Transaction Mechanism Implementation Using Patterns

The very first design question that needs to be addressed is whether the LtB transaction is to be supported by the blockchain infrastructure or through software patterns that utilize the existing blockchain infrastructure - thus leading to two general approaches in supporting the LtB transactions:

- (1) Augment the blockchain infrastructure to support a blockchain transaction that spans execution of more than one method.
- (2) Prepare software patterns as an add-on mechanism that utilizes the current blockchain infrastructure but provides support for LtB transaction.

Of course, the first option, option (1), is preferred, but it requires major changes to the blockchain infrastructure requiring major redesign, resources, and time. The second option may be viable provided that the proposed mechanism to support blockchain trade transactions, which involve executions of more than one method, is (relatively) easy to use by a developer when writing a blockchain smart contract. This is the option we pursue as it is less disruptive and is achievable with fewer resources and in a much shorter period.

There are two major issues that need to be addressed. The first one deals with how a long-term transaction is defined/represented using the current blockchain interface of the hosting blockchain, which is addressed in the following subsection. The second one is on finding a mechanism to support LtB transactions and their properties, which is the topic of the next section.

2.5 Representing LtB Transactions in Smart Contracts

A blockchain transaction is created by an execution of a smart contract method that writes to the ledger. The transaction method generally uses temporary variables/objects, declared within the scope of the method and existing only for the duration of the method's execution. When the method exits its execution, the result of the method's computation is available outside of the method's scope only through:

- A value returned by the method, which could be an object with attributes that could be references to other objects.
- Objects stored on the ledger by the method.

However, the temporary data is not exposed for access outside the scope of a method, and the return value or objects written to ledger are available outside the scope of the method only upon exit from that method. Hence the transaction ACID properties, including the isolation property (no dirty reads), are supported. The transaction writes to the ledger are only applied to the ledger when the transaction is successfully committed, i.e., when the consensus algorithm approves the block containing the transaction and that block is appended to the ledger chain.

An LtB transaction should be similar. The methods of the LtB transaction need to represent and access a shared state of the transaction as it is affected by the execution of the methods of the LtB transaction. And once an LtB transaction begins, this temporary state needs to persist from one execution of the LtB transaction method to another. Furthermore, any writes to the ledger, performed by the transaction, should be applied to the ledger only upon the successful commit of the transaction.

The previous section concluded that a blockchain transaction is a subset of the methods of the smart contract, wherein the system enforces the properties of the long-term blockchain transactions. To enforce the properties, the system must know when a transaction starts and when it ends, which can be represented in two ways:

1. As in a DB system, the transaction begin and end are signaled explicitly, by the application to the LtB transaction mechanism, using two explicit smart contract methods, one to begin an LBT transaction and one to end it.
2. There is no explicit signaling to begin or end a transaction. Instead, the mechanism itself determines when a transaction begins and ends by keeping track of the internal state of the transaction. Thus, any call to one of the smart contract methods of a long-term transaction initiates a transaction, if the current state indicates that there is no transaction. If a call to a method of a long-term transaction results in a transition to a state that indicates an end of the transaction, either abort or commit, then the method exits while indicating to the caller that the transaction has ended, i.e., was either aborted due to an exception or was committed.

We choose the first method of explicit signaling, wherein two methods of the long-term transaction mechanism are used for such signaling, one method to begin the transaction and one to end it. The reasons for selecting this option are:

- This option is analogous to DB systems in which an application signals explicitly to the DB system the beginning and ending of a transaction.
- When a transaction starts, initialization of the transaction state is likely needed an *LtB-transaction-begin* is an obvious choice for such initialization. Similarly, when a transaction ends, with a commit or abort, again, transaction completion activities are needed, and the *LtB-transaction-end* method is the natural place for it. In option (2), these activities would need to be represented by two methods, one for initialization and one for the completion of a transaction, and these methods would be invoked by any method that either starts or ends a transaction.
- To limit access to the methods of the transaction only to the participants of that transaction, the mechanism must know who the participating actors are, and this information is conveniently provided to the mechanism as a parameter to the method that begins the transaction. If the option (2) were taken, difficulties would arise in how to communicate to the transaction mechanism which of the smart contract actors are valid participants in an LtB transaction.

Definition: Long-term Blockchain (LtB) transaction is a subset of the smart contract methods such that:

- The transaction methods are *independent* in that do not refer to any external data/objects or that they do not invoke any external methods. External objects or methods are those that are declared outside the scope of the LtB transaction methods.
- The transaction methods include two special methods:
 - o LtB-transaction-begin method that is invoked by the application to initiate a transaction, such that the first input parameter to the method is the list of identities of actors who are valid participants to the transaction.
 - o LtB-transaction-end method that is invoked by the application to complete transaction, wherein:
 - The first input parameter indicates whether the transaction completion is commitment or abort of the transaction.
 - The method may return a value that is defined by the developer.
- The transaction state, which is deemed to persist across execution of the transaction methods, is represented by an object with attributes, such that each attribute has the getter and setter methods. The transaction state persists only for the duration of the transaction.

Semantics of the transaction model correspond to the semantics of the blockchain transaction:

- The state of an LtB transaction prior to the transaction commit is only accessible to the methods of the LtB transaction.
- Any writes to the ledger must satisfy the ACID properties.

The developer view of current blockchain transaction is that of an invocation of a smart contract method with input parameters. Only when the method completes, it returns a value (output parameters) to the caller, and the transaction ledger writes may be accessed.

The LtB transaction behaves similarly. Invoking the LtB-transaction-begin method initiates the transaction and then the methods of the transaction may be invoked. The methods may have input parameters, but the state of the transaction is not available outside the scope of the LtB transaction. Furthermore, ledger writes made by the methods of the LtB transaction are available outside the scope of the transaction only after the LtB-transaction-end method is executed.

Thus far, we proposed abstractly how the LtB transaction is represented, but not how it is used by the developer or how it results in the final smart contract that is deployed on the blockchain. In the following section we describe our approach to minimizing the developer's responsibilities in writing the LtB transaction methods.

2.6 Minimizing Developer's Responsibilities

From the implementation perspective, the developer can *apply patterns* on smart contracts, patterns such as those that facilitate access control [HeartBank 2018] or patterns that implement the LtB transaction mechanism, which is our objective. Applying the patterns means that once the smart contract methods are written, they are augmented with patterns that provide the desired functionality, such as access control. However, instead of relying on the developer to insert appropriate patterns in appropriate points in the smart contract methods, we automate the tasks by delegating them to a tool. The developer writes the

methods of the smart contract and methods of a LtB transaction as per usual, using the current tools, but with the following constraints or requirements:

- The LtB transaction methods must be *independent*, that is, each LtB transaction method may refer to only methods and objects within the scope of the LtB transaction methods and not to any methods or objects that are declared externally, that is outside the LtB transaction methods.
- The state of an LtB transaction is declared as a specially marked object with members, such that each member has only a getter and a setter method. The LtB transaction methods access the state object only using the provided setter and getter methods. Each LtB transaction method has access only to the locally declared methods and variables and the state of the object that is accessed only through its setter and getter methods. The state object needs to be declared identically in each of the LtB transaction method.
- The transaction must be started and ended by invocation of the *LtB-transaction-start* and *LtB-transaction-end* methods, respectively. The two methods are not provided by the developer, but rather by the LtB transaction mechanism.

The LtB transaction mechanism is achieved by augmenting the LtB transaction code with the patterns that implement the LtB transaction mechanism. However, to achieve that, the developer must identify which of the smart-contract methods belong to the LtB transaction as well as which object, with its members and getter and setter methods, represents the state of the LtB transaction. This is required so that prior to the compilation, the smart contract methods are augmented with patterns that implement an LtB transaction mechanism, which can be achieved in various ways, such as:

- The smart contract methods are augmented using a pre-compiler and the developer uses special pre-compiler commands to identify the LtB transaction methods and the object to represent the state of the LtB transaction.
- Before compilation, the developer invokes a tool and provides it with information on the source code containing the methods of smart contract, which of the methods belong to the LtB transaction, and which object serves as an LtB transaction state.

The smart contract methods are augmented, using pre-processing, with appropriate patterns that implement a LtB transaction mechanism, while ensuring that the transaction state persists across the execution of an LtB transaction. Only after the LtB transaction patterns are augmented can be the smart contract compiled and deployed. The developer provides the following information as input parameters to the pattern augmentation process:

- Reference to the source code containing the smart contract methods that need to be augmented with patterns.
- List of the LtB transaction methods.
- List of identities of actors who are valid participants to the LtB transaction.
- Information about the blockchain on which the successfully compiled smart contract should be deployed.

In other words, the developer need not worry about which design patterns to include at which places in the methods of a smart contract. The developer invokes a tool that modifies the smart contract by including appropriate design patterns automatically. The developer simply needs to let the tool know where the smart contract source code is, which of the methods of a smart contract are part of an LtB transaction, and identities of actors who may access the methods of the LtB transaction – and this constitutes the minimal information needed by any mechanism as the mechanism needs to know which smart contract methods are a part of an LtB transaction and which actors may participate in the transaction.

As described above, the developer proceeds with writing the smart contract methods, including the LtB transaction methods, that update the ledger as per usual, i.e., without any worries about how transactional properties are enforced. Although the state object and its getter and setter methods are declared by the developer only once, each of the transaction methods needs to access it. Thus, before compilation and deployment of the smart contract, the LtB methods need to be augmented with appropriate patterns by the LtB tool facilitate the ACID properties, access control to the methods of the LtB transaction, and privacy. In the following section, we describe how the transactional properties are facilitated by augmenting the smart contract created by the developer with patterns that implement the LtB transactions and their properties.

It should be noted that a transaction may be represented in equivalent but different ways and still be processed by the tool, or its equivalent, resulting in correct augmentation of the smart contract with patterns implementing the mechanism. For instance, instead of the tool to perform pre-processing, a pre-compiler may be used. The developer identifies the LtB transaction methods by special codes recognized by the pre-compiler, also referred to as a pre-processor. For instance, the developer may include all LtB transaction methods in one section of the program identified, or “bracketed”, by special pre-compiler instructions, such as “+Begin LtB transaction” and “+End LtB transaction”. The pre-processor can identify the LtB transaction methods by recognizing the pre-processing instructions using special codes and the pre-compiler amends the contract with appropriate patterns.

3 ALTERNATIVE MECHANISMS TO SUPPORT LTB TRANSACTIONS

We make observations on how existing blockchain systems affect the design choices on the LTB transaction mechanism and then discuss how the private workspace cache is supported as it is needed in any of the alternative mechanisms that we discuss. Finally, we present different alternative architectures that can be used for the LtB transaction mechanisms. Evaluation of the alternative mechanism is performed in the next section.

3.1 Blockchain Properties: Implications on Mechanisms

We make a few observations about blockchain properties and discuss their implications on the design of the mechanism to support the LtB transactions.

a) ACID Properties Enforcement

One of the features that distinguishes blockchains from DBs is immutability – there are only two ledger operations, read and insert/write – update and delete are not available. The immutability property has a consequence for any blockchain transactional mechanism – the mechanism cannot “undo” a transaction operation by restoring the item’s old image/value before the update, as may be performed in a DB. Thus,

any mechanism supporting ACID properties of an LtB transaction may use only optimistic concurrency control approaches, in which the transaction's writes to the ledger are made in some private workspace and, only if the transaction is committed are its updates made to the ledger itself as a part of the commitment phase of the transaction. The main issue to overcome is that the LtB transaction mechanism can only use the native blockchain transaction support that enforces a blockchain transaction as ledger reads and writes of only one smart contract method, wherein the LtB transaction includes writes of several methods of the smart contract. Thus, the LtB transaction mechanism must have the following properties:

- All ledger reads and writes by the LtB transaction methods must be recorded in some private workspace, to which we shall refer as an LtB transaction cache, or simply as a *cache* for short. Only when the *LtB-transaction-end* method is called, that method needs to propagate all reads and writes, performed on the ledger data in the cache, from the cache to the ledger itself as a part of execution of that method. That is, all cached reads and writes performed by methods of the LtB transaction on the cache are replayed by the *LtB-transaction-end* method on the ledger itself.
- The LtB transaction state, which is shared by all methods of the LtB transaction, must also be hosted in the cache as the state of the transaction must be accessible by all methods of an LtB transaction.

b) Access Control for LtB Transaction Methods

To enforce access control to the methods of the LtB transaction, we adopt a simple approach by ensuring that any actor who invokes an LtB transaction has credentials to do so, i.e., that the actor is a valid participant to the LtB transaction. This approach to the access control will be adopted for all alternatives discussed in the next subsection.

c) LtB Transaction Privacy

Although the LtB transaction mechanism does provide access control to the methods of the LtB transaction as described above, such access control does not prevent visibility to the ledger data written by the LtB transaction if any actor, perhaps participating in smart contract but not participating in the LtB transaction, can read data stored in the workspace hosting the cache. Thus, an LtB transaction mechanism needs to prevent such an access, and how and to what level the privacy is supported by the various alternatives will be a part of the evaluation of the alternative approaches to the LtB transaction mechanism, which is performed in the next section.

3.2 Private Workspace Mechanism

To support ACID properties, any reads and writes to the ledger before the transaction commits must be made using the cache provided by the private workspace instead of manipulating reading/writing the ledger itself. Only when the LtB transaction ends, all reads and writes must be propagated to the ledger by one smart contract method execution. To facilitate that, the tool augments the smart contract methods with the following patterns.

- The method *LtB-transaction-begin* is provided as a pattern that prepares the cache and seeds it with the initial data obtained from the ledger. Which data is used to seed the cache is determined by the static analysis of the LtB transaction.
- The cache must persist across executions of the LtB transaction methods and must be accessible by them.

- Each read or write to the ledger by any of the LtB transaction methods must be replaced with a pattern to make that read or write using the cache instead.
- The method *LtB-transaction-end* is provided as a pattern that propagates all transaction reads and writes, made using the cache, to the ledger itself.

It should be noted that the cache stores all ledger reads and writes made by the LtB transaction methods. Furthermore, the *LtB-transaction-end* method does not only propagate/replays all writes from cache to the ledger itself, but it also replays reads, i.e., it re-issues all reads that the LtB transactions made on the ledger. This is required, as some blockchains, such as Hyperledger fabric, need to know both read and write sets of a blockchain transaction to ensure consistency/correctness of the transactions.

Also to be noted is that, due to the dynamic nature of the LtB transaction methods, they may perform reads on the cache that cause cache faults that need to be serviced from the ledger, i.e., the LtB transaction must be augmented by the tool to invoke a method in the smart contract that reads the ledger and stores the read-value in the cache and thus services the cache fault. Consequently, the tool must augment the smart contract with such a method to service a cache fault for an LtB transaction method.

Before we proceed with presenting the alternative mechanisms, we need to discuss options available for hosting the cache as that will affect the options available for the alternative LtB transaction mechanisms. Before the LtB transaction ends, all ledger reads and writes must be done using a cache before the transaction ends. The cached reads and writes must be propagated, “replayed”, from the cache to the ledger itself as a part of execution of *the LtB-transaction-end* method. And the same must apply to the data structure containing the state of the transaction – the state must be kept in the cache as it must persist from one invocation of an LtB transaction method to another invocation of an LtB transaction method. We shall argue in the following that how the private workspace mechanism, i.e., the cache, is implemented depends on how the cache is hosted. And that also affects the alternative mechanisms available for the LtB transaction mechanisms. Thus, we first explore the issues that lead to the different approaches to hosting the cache, which in turn are used to determine the alternative mechanisms that will be discussed later. There are two main approaches in hosting the cache and these are discussed in the following subsections.

3.2.1 Cache Hosted on Blockchain Data Structure

This approach is available only if the blockchain infrastructure provides a data structure that persists across executions of a multiplicity of methods of a smart contract. For instance, Hyperledger fabric provides for *private data* that is accessible only to designated actors. Not only does the data persist across the multiplicity of executions of smart contract methods, but privacy is assured in that it the private data may be accessed only by those designated actors.

Another example is Ethereum that provides for memory variables, which can be declared within the scope of a smart contract, that persist across multiplicity of execution of the smart contract methods. However, such variables are accessible to any methods of the smart contract whether they belong to the LtB transaction or not. Thus, privacy is not provided as all methods of the smart contract, regardless of whether they belong to the LtB transaction, can access the cached data.

3.2.2 All Hosted in One Smart Contract

If the blockchain infrastructure does not provide for data structures that persist data across executions of smart contract methods, then there is no alternative but to host the private-workspace cache on the blockchain ledger itself. Below we discuss the case when the cache is hosted within the scope of the smart contract that has the LtB transaction, while in the next subsection we discuss the case when the cache is hosted within the scope of another smart contract.

- Once the *LtB-transaction-begin* is executed, any ledger reads or writes performed by the LtB transaction methods are done on the ledger, but in locations known only to the transaction methods, by performing mapping from actual keys to temporary keys. Thus, once an LtB transaction starts, any read or write, using the key k_{actual} , such as in $read(k_{actual},)$ or $write(k_{actual}, v)$, is transformed to a read or a write using a temporary key k_{temp} , so that the actual read and write become $read(k_{temp})$ or $write(k_{temp}, v)$, respectively. The LtB transaction mechanism must provide for translation of the actual keys to the temporary keys.
- When the *LtB-transaction-end* method is invoked and executed, all ledger reads and writes that were made on the cache using temporary keys are propagated from the cache to the ledger using the actual values of keys. Thus, the LtB transaction mechanism must keep track of the reads and writes performed to the cache (on the ledger using the temporary keys) so that they could be replayed on the ledger using the actual keys.

In the next section, which evaluates alternative LtB transaction mechanisms based on the cache hosting method, we will show that the ACID properties and access control are supported. However, privacy is more challenging as any actor has access to any ledger object/value as long as the actor has the object's key used to store the object on the ledger. And, if an attacker knows the location of the smart contract address on the ledger, then the attacker has access to the smart contract code and may be able to infer from the code how the keys are generated and hence breach privacy.

An obvious approach to privacy, when the cache is hosted by the smart contract hosting the LtB transaction, is to use the public-key cryptography to protect the cache and we describe two approaches that use cryptography:

- *All writes to the cache are encrypted*: Cryptography is used on any ledger data read or written. (We refer to such an approach as *Smart contract hosting the cache with data cryptography*, or *SC-data-crypt* for short in tables below.) It is a secure method that ensures privacy, however, at the cost of the cryptographic overhead and dealing with the issues of the private-key management. If the transaction is aborted, the cached ledger writes are simply left in the cache in the encrypted form and thus un-viewable.
- *All in one smart contract with cryptography on keys*: This is another way to increase the difficulty of the attack using cryptography. Cryptography is used on the key used to read/write the cache data on the ledger. (We refer to this method as *SC-key-cryp* in figures/tables.) When an LtB method performs a ledger read, $read(k_{actual})$, or ledger write, $write(k_{actual}, v)$, when the LtB mechanism transforms the value k_{actual} to k_{temp} , the k_{temp} values are encrypted thus making it more difficult for the attacker to find the stored object. In comparison to the previous method that uses the cryptography, this method encrypts the keys only, i.e., encryption/decryption is performed on the fixed length keys, while the former method encrypts/decrypts all data value associated with the

read/write and hence it depends on the length of the data read/written and not the length of the key used to store the data value.

3.2.3 Cache and LtB Methods in a Separate Smart Contract

Yet another architectural alternative is to use a separate smart contract to host the cache and LtB transaction methods. In addition to the main smart contract that contains all methods that are not part of the LtB transaction, while a side contract is used to host the cache and the LtB transaction methods. Although cryptography is not involved, the attacker would not only need to have access to the code of the smart contract but would need to use the code of the smart contract to infer the location of the smart contract hosting the cache, plus using the code of the cache to infer the values of the temporary keys used to store objects in the cache. There are two options in hosting the side contract:

- *Cache and LtB methods in a separate smart on mainchain:* Both smart contracts are hosted on the same chain, referred to also as the *mainchain*. (We refer to this method as *main+main* in tables.)
- *Cache and LtB methods in a separate smart on sidechain:* The main contract is hosted on the mainchain, while the LtB methods and the cache are hosted in a separate contract that is deployed on a sidechain. (We refer to this method as *main+side* in figures/tables.)

3.3 Alternative LtB transaction Mechanisms

The previous discussion guides the selection of the design alternatives that we explored for supporting LtB transactions, with each one providing the ACID properties and access control, but each one with varying degrees of privacy expressed in terms of effort on the attackers' part to breach the privacy. We used them to derive the following alternative mechanisms to support the LtB transactions.

We first describe the base case, used for comparison purposes, when there is no LtB transaction support. We then describe two cases when the cache is hosted by the native blockchain data structure that persists across executions of the methods of a smart contract, wherein in one case the blockchain data structures also ensures privacy, such as in the *private data* type in Hyperledger fabric, while in the second one there is no privacy as objects in the data structure may be viewed by any actors.

We then describe the two mechanisms in which the cache and the LtB transaction methods are hosted in a separate smart contract, wherein in one case both smart contracts are on the same chain, while in the second case, the two smart contracts are hosted on two separate chains. Finally, the last two mechanisms use cryptography to support privacy. The alternative mechanisms are now described.

- *Smart contract with no LtB transaction mechanism – base case:* There is no support for the LtB transaction mechanism. We use the base case for comparison with other approaches that provide LtB transaction support using the alternative mechanisms. (We refer to this method as *no-LtBx* in figures/tables.)
- *All in one smart contract with cache in memory variables:* All methods, transaction or not, are deployed in one smart contract, with the cache being supported by a data structure provided by the native blockchain infrastructure, data structure that persist across executions of a multiplicity of executions of smart contract methods but without privacy, such as memory variables in Ethereum (We refer to this method as *SC-all-mem* in figures/tables.) We do not consider in the following

discussion the use of the Hyperledger's *private data* for hosting the cache and LtB methods as we did not find any suitable ways to estimate the cost of the LtB mechanism.

- *All in one smart contract with cache in private data structure*: All methods, transaction or not, are deployed in one smart contract, with the cache being supported by a data structure provided by the native blockchain infrastructure, data structures that persist across executions of a multiplicity of executions of smart contract methods and provides for privacy, such as private data in Hyperledger fabric. (We refer to this method as *SC-all-priv* in figures/tables.)
- *Cache and LtB methods in a separate smart contract on mainchain*: There are two smart contracts on the same blockchain: The main smart contract containing the methods that are not a part of the LtB transactions and thus invoke the LtB transaction methods. The second smart contract, also deployed on the same mainchain, hosts the cache and the LtB transaction methods. (We refer to this method as *main+main* in figures/tables.)
- *Cache and LtB Methods in a separate smart contract on sidechain*: This method is similar to the above method (*main+main*), except that the smart contract hosting the cache and the LtB transaction methods is deployed on a sidechain instead of the mainchain. (We refer to this method as *main+side* in figures/tables.)
- *All in one smart contract with crypto on keys*: All smart contract methods and the cache are deployed in one smart contract with cryptography used to encrypt the keys of cached objects. (We refer to this method as *SC-key-cryp* in figures/tables.)
- *All in one smart contract with crypto on cached data*: All smart contract methods and the cache are deployed in one smart contract with cryptography used on all cached data. (We refer to this method as *SC-data-cryp* in figures/tables.)

In all the above cases, the LtB transaction tool performs the amendments of the smart contract with LtB transaction mechanism patterns as described in the subsection on the private-workspace mechanism, wherein the patterns differ depending on the architectural alternatives described above.

4 EVALUATION

To evaluate the above-proposed alternative mechanisms and hence our overall approach, we examine each mechanism to determine whether the required ACID properties and access control are enforced, and how the mechanism compare, relative to each other, in terms of the privacy. In the first two subsections, we show that the ACID properties and access control are fully supported. In the next subsection we discuss privacy issues for each of the alternative architectures. Of course, supporting the LtB transactions is not without overhead cost – thus, overhead costs are discussed in the last subsection that compares the overhead costs incurred by the alternative mechanisms.

4.1 Enforcing ACID properties

Each of the alternative mechanisms enforces ACID properties as all reads and writes made by the LtB transaction on the private workspace are also made by the execution of the *LtB-transaction-end* method that is invoked by the application to successfully end an LtB transaction. As all ledger reads and writes are performed by one smart contract method, the *LtB-transaction-end* method, the ACID properties of that

method are ensured by the native blockchain transaction mechanism and hence the LtB transaction ACID properties are supported.

4.2 Enforcing Access Control

To enforce access control on the methods of an LtB transaction, we adopt a simple approach by ensuring that actors, who invoke the LtB transaction methods, have valid credentials to do so. When an LtB transaction is started by a call to the *LtB-Transaction-begin* method, the input parameters to that call also include the list of identities of actors who participate in the transaction. Once that is supplied, the mechanism enforces the access control by ensuring that when any LtB transaction method is invoked, the identity of the invoker matches one of the identities provided in the list of identities that are participants to the transaction. To enforce the access control, each method is augmented with a pattern to enforce access control [HeartBank 2018]. This approach to access control is adopted for each of the architectural alternatives discussed below. Thus, access control property is assured by each of the alternative mechanisms.

4.3 Enforcing Privacy

Recall that privacy here is for data written to the ledger by the methods of the LtB transaction. We already discussed implications on privacy by LtB transaction mechanisms in the sections 3.2. In this section we compare the alternative mechanisms in terms of supporting privacy. We simplify the comparison by ranking the mechanisms in relative terms of the effort required to breach privacy. When comparing two methods, we determine which one requires more or less effort than the other method to breach privacy. The result of ranking of the methods, from the best to the worst case, is shown in Table 4.1. However, below we shall discuss them in reverse order of presentation in the table, i.e., from the one requiring the least effort in breaching the privacy to the most.

Clearly, the base case requires the least effort to breach privacy as there is no support for LtB transactions, the case of *no-LtBxa*, and hence no support for privacy. Hence it is the worst case appearing at the bottom of the table (#7 in the table).

The second worst mechanism (#6 in the table), in terms of privacy, is for the case of *SC-all-mem*, i.e., when all methods are in one smart contract with the cache being hosted by the persisting data structure provided by the blockchain infrastructure, but without privacy, such as *memory variables* in Ethereum. The attacker would only need to know the address of the smart contract and use de-compilation to infer keys used to store data in the cache, i.e., in the memory variables and thus breach privacy.

The third worst mechanism (#5 in the table) is for the case of *main+main*, i.e., when the cache and LtB methods are in separate smart contract on the mainchain. An attacker would need an address/location of the main smart contract on the ledger to use de-compilation to reveal the address of the smart contract hosting the cache and the LtB transaction methods. And then again use de-compilation to reveal the values of the keys used to write ledger objects, thus breaching privacy.

The fourth worst mechanism (#4 in the table) in providing privacy is the case of *main+side*, i.e., when the cache and LtB transaction methods are in a separate smart contract on a sidechain. The attacker would apply the same technique as in the previous case with the following additional challenges that the attacker would have to overcome:

- The attacker would have to discover the location of the smart contract, which hosts the cache and the LtB methods, on the sidechain.
- Furthermore, the attacker would also have to obtain access to the sidechain.

The next two mechanisms, i.e., the third and second in ranking, in addition to effort expended in the previous methods, attack would require breaking the cryptographic methods, which we assume require much more effort than the effort to breach previously discussed mechanisms. The two mechanisms are *SC-key-cryp* and *data-cryp*. The difference between them is that in the first case it is the key that is encrypted, which is of fixed size. In the second case, encryption applies to any object written to the ledger with the cost being dependent on the data-length. Consequently, the latter (all data encrypted) is harder to break than the former (encrypted key).

Finally, the *Blockchain Private Data Used for Cache (CS-all-private)* mechanism is assumed to require the most effort to breach privacy, because the cache is hosted in a data structure that is provided by the blockchain infrastructure itself. More importantly, if privacy for the blockchain infrastructure is breached, such a breach is applicable to each of the alternative mechanisms under consideration. Thus, we assume, rightly or wrongly, that this mechanism provides most privacy. In the following subsection, we shall examine the relative overhead execution costs of the alternative mechanisms.

Table 1: Ranking of mechanisms in terms privacy protection (from best to worst)

Rank	Alternative Mechanism
1.	Blockchain persisting data structure with privacy hosting the cache (<i>CS-all-private</i>)
2.	All in one smart contract with crypto on cached data (<i>SC-data-cryp</i>)
3.	All in one smart contract with crypto on cache key only (<i>SC-key-cryp</i>)
4.	Cache and LtB methods in separate smart contract on sidechain (<i>main+side</i>)
5.	Cache and LtB methods in a separate smart contract on mainchain (<i>main+main</i>)
6.	All in one smart contract with cache hosted in memory variables (<i>SC-all-mem</i>)
7.	Smart contract with no LtB Transaction mechanism (<i>no-LtBxa</i>)

4.4 Overhead Costs of Alternative Mechanisms

In this subsection, we compare the relative execution costs of alternative mechanisms. As our assumed environment is that of long-term transactions, comparing the total delays of long-term transactions without and with an LtB transaction mechanism is not appropriate as the overhead delays due to the LtB transaction mechanism would be insignificant in comparison of the long-term delays of the transaction activities itself. Instead, we determine the cost of executing the patterns inserted into the smart contract methods to implement the LtB transaction mechanism. We first discuss the method used to determine the overhead costs and then we present and review the results of evaluation.

4.4.1 Cost Estimation Method

To determine the cost of executing patterns for each of the alternative mechanisms, we utilize the Ethereum blockchain tools that enable estimation of the cost of execution of smart contract methods written in the Solidity language. We use Remix compiler that provides for compilation of a Solidity smart contract method into executable code and estimates the total gas cost of executing a smart contract method.

Solidity is a high-level language for writing smart contracts for the Ethereum blockchains and estimating the cost of executing a smart contract method. Whenever a user application invokes a smart contract method, the cost of execution of that method is charged using gas units, wherein the cost of a unit of gas fluctuates and depends on the load on the system at any particular point in time: Increased demand for executing smart contract methods by users, means increase in the dollar cost per unit of gas. For each execution of a smart contract method, a user needs to commit maximum amount of gas that may be consumed, “burnt”, by executing smart contract method’s instructions: If the actual gas cost of executing the instruction of the method exceeds the gas amount committed by the user for the execution of that method, the method execution is aborted, and the user loses the committed amount of gas. Thus, the Remix compiler can be used to estimate the gas requirements for method so that the user would have an estimate of how much gas to allocate for executing a smart contract method.

We use the Remix compiler to measure the gas cost of execution of each method of the smart contract with and without the patterns to implement the LtB transaction mechanism and thus estimate the relative overhead cost of providing an LtB transaction mechanism when compared to executing the LtB transaction methods without the LtB transaction mechanism. For each alternative mechanism, except for the case of CS-all-private, we performed two measurements using a smart contract that contains the following methods before amending them with patterns to implement the LtB transaction mechanism:

- A simple method that is not a part of the transaction. The method writes a string “start” to a memory variable and returns.
- Two methods that are a part of an LtB transaction:
 - Method *m1(x1: objectSizeX)* ... the method writes to the ledger an object of size X bytes.
 - Method *var keyX2 = m2(x1: objectSizeX, x2:objectSizeX)* ... the method reads the object written by the method m1; it then writes it back to the ledger again as a new object and returns the key, *keyX2*, of the newly written object.

Each of the two methods was written in the Solidity language and compiled using the Remix compiler that, in addition to the compiled code, also provides the estimate of the gas cost to execute the method. The gas costs of the methods, expressed in GWei units, are then added up to produce the total gas cost execution. However, as the case of *CS-all-private* uses the persisting blockchain data-structure with privacy to host the cache and Ethereum data does not provide such a data structure, we were unable to use the Remix compiler and estimate the overhead cost for the *CS-all-private* case.

To estimate the gas costs when the LtB transaction mechanism is implemented, we first use pre-processor to insert into the smart contract methods patterns that implement the LtB transaction mechanism. The inclusion of the LtB transaction patterns means that LtB transaction methods are augmented with inclusion of *LtB-transaction-begin* and *LtB-transaction-end* methods and amending the methods that read and write the ledger with patterns to write to the cache instead. We also calculate the gas cost for (i) propagating all reads and writes from the cache to the main ledger by the *LtB-transaction-end* method and (ii) for encryption/decryption. As the method *m1* first writes the object, passed as a methods parameter, to the cache and then the method *m2* reads to write a new copy of the object to the ledger, there are no cache faults. Finally, we perform the gas-cost calculation for each of the alternative

mechanisms to support the LtB transaction mechanism in addition to the base case when there is no LtB transaction mechanism:

- *No LtB Transaction Overhead* ... Gas-cost was estimated using the Remix compiler for the smart contract that contained the methods of the LtB transaction before amendments with patterns that provide the supporting LtB transaction mechanism.
- *With LtB transaction Overhead* ... For each of the alternative mechanisms, the smart contract was amended with the patterns that provide the LtB transaction mechanism. After the amendment, the LtB transaction includes two additional methods, the *LtB-transaction-begin* and *LtB-transaction-end* methods. In addition, the method *m2* was augmented with a pattern that writes to cache instead to the ledger. Recall that the *LtB-transaction-end* method replays all reads and writes stored in the cache to the ledger itself, i.e., each read or write to the ledger that was stored in the cache is replayed, re-issued, to the ledger.

4.4.2 Cost Estimates

Table 2 shows the gas-cost estimates for each of the LtB transaction mechanisms. Each row is for a specific case, wherein the case labels, appearing in the first column of each row, identify the alternative mechanisms used to implement the LtB transaction mechanism. The labels are shown below:

1. **No-LtBxa...** One smart contract with no LtB transaction mechanism
2. **SC-all-mem...** All LtB transaction methods and cache are in one smart contract w cache in memory vars.
3. **SC-key-cryp...** All in one smart contract with crypto on the keys only
4. **Main+side...** Cache+LtB Methods in smart contract on a sidechain
5. **SC-data-cryp...** All in one smart contract with crypto (all ledger data encrypted)
6. **SC-data-cryp...** One smart contract with Data-crypto

The table shows the gas cost for each of the alternative mechanism. Graphical representation of the table data is shown in Figure 1. The first column of the table contains the label identifying the LtB mechanism (labels are shown above). The subsequent columns show the cost estimates for the alternative mechanisms when the LtB transaction reads and writes a ledger object of size $X = 75, 512, 1024$, and 1875 Kb. Note that for current Ethereum, the 1875 Kb is the maximum block size that can be used for recording one transaction on the ledger [Ethereum's New 1MB Blocksize Limit, 2021].

Table 2: CPU Processing Cost Estimates Determined Using Remix Compiler

Label	7kb	512 kb	1024 kb	1875 kb
<i>No-LtBxa</i>	4545000	31027200	62054400	113625000
<i>SC-all-mem</i>	9090582	62054982	124109382	227250582
<i>main-main</i>	9319500	63621120	124113000	227254200
<i>main-side</i>	9405000	64204800	124109382	227250582
<i>cs-key-cryp</i>	9724782	66359982	132715182	243004782
<i>cs-data-cryp</i>	9763650	66653184	133306368	244091250

It should be noted that the gas-cost calculations exclude any overhead costs due to the execution of the blockchain work that is not the direct execution of the methods. For instance, the gas costs exclude the cost of data exchange amongst the network nodes due to replication, the cost of endorsement/approval of individual transactions, such as is done by Hyperledger, or the cost of the consensus algorithm. Furthermore, the cost of the cache and LtB methods in the *Separate smart contract on a sidechain (main+side)* method, we estimated the cost of the sidechain processing to be equivalent to that of executing on the mainchains, which is likely to be overestimated as sidechain processing is usually considered for lowering the cost of processing by performing it on a much less-expensive sidechain.

Figure 1. CPU Processing Cost Estimates

The next two lowest costs are for the methods that use two separate smart contracts, with the main smart contract containing the non-transaction methods, while the second smart contract hosting the cache and the LtB-transaction methods. The difference between the two mechanisms is that in the *main+main* case, both contracts are on the mainchain, while in the second case, *main+side*, the smart contract hosting the cache and the LtB transaction methods is deployed on the sidechain. Of course, the *main+side* mechanism is more expensive due to the cost of invoking the LtB transaction methods on the sidechain which is more costly than when the cache is hosted on the main chains as is done in the *main+main* case. However, if the cost of sidechain processing is much cheaper than on the mainchain, then the *main+side* case may be more cost effective.

are less expensive cryptographic methods, albeit providing less privacy, that may be used and which may be less expensive.

General Observations

The cost estimate show that, in Ethereum, the cost of reading and writing the ledger or the memory variables comprise the dominant cost. This is not surprising as Ethereum documentation [Opcodes for the EVM 2023] corroborates that the ledger writes are most expensive, closely followed by the cost of the memory variables. The cost of storage on a local stack is much cheaper, albeit the stack size is capped at a maximum that, if exceeded, leads to the abort of the smart contract method. Furthermore, a stack variable does not persist across executions of smart contract methods.

The costs also show that supporting LtB transactions is not cheap as it more than doubles the cost of execution of the base case that does not provide support for LtB transactions. Furthermore, the cost of each mechanism is directly proportional to the size of the ledger data read/written. For Ethereum public blockchain, the cost of processing methods of a smart contract is expensive as the number of network nodes is high. For permissioned blockchains that may have smaller number of network nodes, the cost of processing may be significantly lower.

5 NESTED TRANSACTIONS

A nested transaction is declared within the scope of another, outer transaction, that already started its execution and initiates execution of the inner transactions [Nested transaction, 2019]. Nested transactions have been studied intensively in the context of the DB systems, wherein the usual transaction semantics are applied: An outer transaction commits only if all its inner transactions are finished successfully with their work. Furthermore, the concept of an inner transaction may be applied recursively, thus achieving nested multi-level transactions. When referring to nested transactions, we refer to multi-level nested-transactions and the objective of this section is to describe how the LtB transaction is extended to support nested LtB transactions.

We first describe the overall approach of supporting nested transaction using a standard layer-n approach when extending the blockchain functionality. We then briefly overview how a nested transaction is declared and then how it is augmented with patterns to support nested LtB blockchain transactions.

5.1 Layered Approach in Supporting Nested Transactions

Recall that to create an LtB transaction, the developer:

- Creates the state object of which members are accessed only through the setter and getter methods.
- Creates the methods of an LtB transaction, such that each method has access only to the locally declared objects, which persist for the duration of the methods execution only, and members of the state object that are accessed only through their getter and setter methods, wherein the state object persists across execution of the smart contract methods.
- Identifies the state object and the LtB transaction methods and invokes a pre-compiler that inserts appropriate patterns into the smart contract, patterns that implement the LtB transaction mechanism. If all goes well, the smart contract with augmented patterns is compiled and deployed.

We also examined a number of alternative architectures used to implement the LtB transactions, including options in which the cache and the LtB transaction methods are packaged into a separate smart contract: The main contract contains all methods of a smart contract that do not participate in an LtB transaction, while another smart contract, to which we shall refer to as a transaction smart contract, contains the cache and the methods of the LtB transaction. Thus, the LtB transaction functionality is encapsulated in the transaction smart contract. It should be noted that the transaction smart contract is typically deployed on a sidechain when it is cheaper than the mainchain.

The above approach is an example of using a layer-2 network used to improve blockchain transaction functionality. Layer-1 blockchain network is used to invoke smart contracts that perform work, while the layer-2 network is used to record the proofs of results of the layer-1 network [Gemini 2023]. Recursively applying the approach to support a sub-transaction within a sub-transaction will achieve an LtB transaction mechanism that supports multi-level nested transaction.

5.2 Declaring Nested Transaction

The outer nested transaction is declared as per previous sections. The developer identifies the transaction methods and the state cache data structure so that the smart contract may be augmented with appropriate patterns. If an outer transaction is nested, i.e., it includes one or more sub-transactions, each of the sub-transactions is appropriately declared as before. The developer prepares and identifies methods of each sub-transaction and the object cache. Thus, by recursively applying the declaration of a transaction within another transaction, multi-level transaction is achieved.

5.3 Multi-level LtB Transaction Mechanism

To augment a smart contract with LtB transaction mechanism, the pre-compiler must augment the patterns for each of the inner sub-transaction first before it augments the patterns of the outer transaction. Of course, this also needs to be applied recursively. In terms of the commitment, 2PC commit is applied for each nesting. That is, an outer transaction may commit only if all of its sub-transactions are prepared to commit – again applied recursively.

For each outer transaction, in essence, there is a 2-Phase Commit (2PC) protocol that is used to prepare its participants for commitment first, before the outer transaction commits and issues the command to commit to its sub-transactions. That is, the outer transaction acts as a coordinator of the 2PC, while each of its sub-transactions acts as a 2PC protocol participants. The coordinator leads the sub-transactions through the first phase of the protocol to prepare for commitment. The outer transaction commits and then issued the commit command to its participants.

If, however, the outer transaction X is an inner sub-transaction itself due to multi-level nesting, it serves two roles: As a coordinator of the 2PC protocol for its sub-transactions and as a participant of the 2PC in a transaction in which it serves as a sub-transaction:

- Transaction X first leads its sub-transactions through the first phase of 2PC and gets participants ready to commit.
- It then acts as a participant of 2-PC as a sub-transaction of its parent transaction.

After receiving the ready-to-commit signals from its sub-transactions, before commit command is issued to its sub-transaction commitment, the transaction X becomes a participant of the 2PC protocol reporting to its coordinator. Only after X receives the commit command from its coordinator can X issue the commit command to its sub-transactions.

We are currently developing a Proof of Concept (**PoC**) to show that the approach is feasible for implementation using current blockchains, such as Hyperledger fabric.

6 RELATED WORK

In this paper we addressed the issue of how to support in blockchains transactions that include executions of a multiplicity of smart contract methods. We showed that in trade of goods and services, there is a need for such a transaction as transaction activities may be long-term, with unpredictable delays and sequencing of inputs to the transaction by various collaborators. To the best of our knowledge, this problem has is yet to be addressed in the scientific literature. Instead, research and industry concentrated on supporting cross-chain transactions, particularly in support of **Non-Fungible Tokens (NFTs)**. The main issue for cross-chain transaction is the transfer of NFTs across the chains to ensure that NFTs are unique across the chains and to ensure the atomicity of transactions involving NFTs. Thus, the issue is how to manage NFTs across participating chains, in particular in transferring an NFT across chains, such as a user on one chain can purchase/trade an NFT using a transaction that has the appropriate guarantees of atomicity, while guaranteeing that the NFT can exist on one chain only [Non-Fungible Tokens (NFTs) Explained 2023, Non-fungible token 2023].

It was already pointed out in a previous section that our approach to implementing the LtB transaction mechanism utilizes the layered network approach as is done for the Ethereum to improve the transaction throughput: Two network layers are used, wherein one executes the “main” transactions that simply record the proofs of work performed by sub-transactions on another network layer. We use the same approach to support multi-level nested transaction: The outer transaction has inner transactions that are executed under the control of the outer transaction, wherein the outer transaction records the results of the inner transactions and uses a commit protocol, such as 2PC protocol, to commit its sub-transactions. The multi-level nesting is achieved by recursively applying the concept of a nested transaction.

Also related is our previous work on supporting the long-term transactions in the context of transforming a BPMN model into the methods of a smart contract [Bodorik 2022]. A BPMN model goes through several transformations, eventually resulting in a smart contract(s) that is deployed on a blockchain – all automatically under the guidance of the developer [Liu 2022]. The BPMN model is extended with the concept of a long-term transaction that represents the transaction’s collaborative activities by the transaction participants. The BPMN model, with the long-term transaction as represented in an extended BPMN, is analyzed, and it proceeds through several transformations that eventually produce the methods of a smart contract(s) that support the long-term BPMN transaction. However, the transformations start with a BPMN model and results in a set of methods of a smart contract(s) such that the BPMN long-term transaction is supported. In other words, there is no mapping of a BPMN long-term transaction into a blockchain long-term transaction that spans multiplicity of block-chain methods - no such concept was described.

7 SUMMARY, CONCLUSIONS AND FUTURE RESEARCH

This paper describes our initial results on supporting nested long-term blockchain transactions that span executions of a multiplicity of the methods of a smart contract, wherein such a transaction is not supported by blockchains. We provided motivation for supporting the LtB transaction and presented a model of our approach for an LtB transaction mechanism using a generalized roll-up techniques used for blockchains. We say we use a rollup layer-2 technique because we utilize an existing blockchain to record the results of an LtB transaction on a blockchain, transaction that is not supported by the underlying blockchain. We say that we use a generalized rollup technique because the work performed on the transaction is not necessarily performed on a layer-2 blockchain, such as a sidechain that is generally much cheaper than the mainchain but can be performed on the mainchain itself.

We presented an LtB transaction model that includes a set of independent methods and use patterns to implement the LtB transaction mechanism. The mechanism enforces the ACID transactional properties and access control to the methods of the LtB transaction, and it also supports privacy. Our current implementation uses sidechains for cache and the state of the transaction that need to persist across executions of the methods of a smart contract. However, our approach may be readily extended to use the mainchain or a cross-chain, instead of a sidechain. Furthermore, using sidechains as layer-2 network may be attractive from the perspective of overhead costs of the LtB mechanism if the sidechain is much less expensive than the mainchain.

Finally, we also described how to support nested long-term blockchain transactions using the 2-layer network approach. Sub-transactions are deployed in separate smart contracts, such that each sub-transaction interacts only with its parent transaction or its sub-transactions. To support the ACID properties, we use the 2PC commit protocol in which the outer transaction acts as 2PC protocol coordinator, while sub-transactions serve as participant in the 2-PC protocol. When multi-level nesting of transactions is used, the transaction may at the same time a parent transaction and a sub-transaction, in which case care must be taken in proper sequencing of the commitment activities across the levels of the multi-level transaction. We are currently working on developing a PoC to show that our approach to support of multi-level long-term transactions is feasible.

However, to use LtB transactions, the software developer needs to examine the application requirements in terms of the collaboration and information exchange and determine the transactions and then tackle the issue of writing smart contracts, once the transaction is defined. Thus, introduction of the LtB smart transaction, although necessary, further complicates the design and development of smart contract. It thus provides further impetus for research into automated generation of smart contracts from models represented at a high level of abstraction. We are currently working on using BPMN modeling to represent the blockchain application requirements and then use automated transformation of the BPMN model into the methods of a smart contract, or smart contracts. Furthermore, the analysis and transformation of a BPMN model into a smart contract(s) should be such that the nested long-term blockchain transactions are automatically identified and transformed into the blockchain nested transaction presented herein. Finally, the smart contracts, which include LtB nested transactions, should be hardened from the security perspective and deployed - also automatically.

REFERENCES

- [1] Chaum, D. (1983). Blind Signatures for Untraceable Payments. In D. Chaum, R. L. Rivest, & A. T. Sherman (Eds.), *Advances in Cryptology* (pp. 199–203). Springer US. https://doi.org/10.1007/978-1-4757-0602-4_18
- [2] Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. <https://www.semanticscholar.org/paper/Bitcoin%3A-A-Peer-to-Peer-Electronic-Cash-System-Nakamoto/4e9ec92a90c5d571d2f1d496f8df01f0a8f38596>
- [3] Marr, B. (2017). A Short History Of Bitcoin And Crypto Currency. *Forbes Innovation and Tech*. <https://www.forbes.com/sites/bernardmarr/2017/12/06/a-short-history-of-bitcoin-and-crypto-currency-everyone-should-read/?sh=283add863f27>
- [4] Szabo, N. (1997). Formalizing and Securing Relationships on Public Networks. *First Monday*, 2(9). <https://doi.org/10.5210/fm.v2i9.548>.
- [5] Szabo, N. (2018). Smart Contracts: Building Blocks for Digital Markets. <https://www.semanticscholar.org/paper/Smart-Contracts-%3A-Building-Blocks-for-Digital-Szabo/9b6cd3fe0bf5455dd44ea31422d015b003b5568f>
- [6] Buterin, V. (2015). A Next Generation Smart Contract & Decentralized Application Platform. <https://www.semanticscholar.org/paper/A-NEXT-GENERATION-SMART-CONTRACT-%26-DECENTRALIZED-Buterin/0dbb8a54ca5066b82fa086bbf5db4c54b947719a>
- [7] Proof of work. (2023). In Wikipedia. https://en.wikipedia.org/w/index.php?title=Proof_of_work&oldid=1138408016
- [8] Taylor, P. J., Dargahi, T., Dehghantanha, A., Parizi, R. M., & Choo, K.-K. R. (2020). A systematic literature review of blockchain cyber security. *Digital Communications and Networks*, 6(2), 147–156. <https://doi.org/10.1016/j.dcan.2019.01.005>.
- [9] Khan, S. N., Loukil, F., Ghedira-Guegan, C., Benkhelifa, E., & Bani-Hani, A. (2021). Blockchain smart contracts: Applications, challenges, and future trends. *Peer-to-Peer Networking and Applications*, 14(5), 2901–2925. <https://doi.org/10.1007/s12083-021-01127-0>
- [10] Vacca, A., Di Sorbo, A., Visaggio, C. A., & Canfora, G. (2021). A systematic literature review of blockchain and smart contract development: Techniques, tools, and open challenges. *Journal of Systems and Software*, 174, 110891. <https://doi.org/10.1016/j.jss.2020.110891>
- [11] Belchior, R., Vasconcelos, A., Guerreiro, S., & Correia, M. (2021). A Survey on Blockchain Interoperability: Past, Present, and Future Trends (arXiv:2005.14282). *arXiv*. <https://doi.org/10.48550/arXiv.2005.14282>
- [12] Saito, K., & Yamada, H. (2016). What's So Different about Blockchain? — Blockchain is a Probabilistic State Machine. 2016 IEEE 36th International Conference on Distributed Computing Systems Workshops (ICDCSW), 168–175. <https://doi.org/10.1109/ICDCSW.2016.28>
- [13] Garcia-Garcia, J. A., Sánchez-Gómez, N., Lizcano, D., Escalona, M. J., & Wojdyński, T. (2020). Using Blockchain to Improve Collaborative Business Process Management: Systematic Literature Review. *IEEE Access*, 8, 142312–142336. <https://doi.org/10.1109/ACCESS.2020.3013911>
- [14] Lauster, C., Klinger, P., Schwab, N., & Bodendorf, F. (2020). Literature Review Linking Blockchain and Business Process Management (pp. 1802–1817). https://doi.org/10.30844/wi_2020_r10-klinger
- [15] Levasseur, O., Iqbal, M., & Matulevičius, R. (n.d.). (2021) Survey of Model-Driven Engineering Techniques for Blockchain-Based Applications. *PoEM'21 Forum: 14th IFIP WG 8.1 Working Conference on the Practice of Enterprise Modelling*.
- [16] Weber, I., Xu, X., Riveret, R., Governatori, G., Ponomarev, A., & Mendling, J. (2016). Untrusted Business Process Monitoring and Execution Using Blockchain. In M. La Rosa, P. Loos, & O. Pastor (Eds.), *Business Process Management* (pp. 329–347). Springer International Publishing. https://doi.org/10.1007/978-3-319-45348-4_19
- [17] López-Pintado, O., García-Bañuelos, L., Dumas, M., Weber, I., & Ponomarev, A. (2019). CATERPILLAR: A Business Process Execution Engine on the Ethereum Blockchain (arXiv:1808.03517). *arXiv*. <https://doi.org/10.48550/arXiv.1808.03517>
- [18] Tran, A. B., Lu, Q., & Weber, I. (n.d.). (2018). Lorikeet: A Model-Driven Engineering Tool for Blockchain-Based Business Process Execution and Asset Management. 16th International Conference on Business Process Management, BPM 2018, Sydney, Australia, September 9–14, 2018, pp. 56–60.
- [19] López-Pintado, O., García-Bañuelos, L., Dumas, M., Weber, I., & Ponomarev, A. (2019). CATERPILLAR: A Business Process Execution Engine on the Ethereum Blockchain (arXiv:1808.03517). *arXiv*. <https://doi.org/10.48550/arXiv.1808.03517>
- [20] Bodorik, P., Liu, C. G., & B Jutla, D. (2022). TABS: Transforming automatically BPMN models into blockchain smart contracts. *Journal of Blockchain: Research and Applications*, 100115. <https://doi.org/10.1016/j.bcr.2022.100115>
- [21] Liu, C., Bodorik, P., & Jutla, D. (2021). From BPMN to smart contracts on blockchains: Transforming BPMN to DE-HSM multi-modal model. 2021 International Conference on Engineering and Emerging Technologies (ICEET), 1–7. <https://doi.org/10.1109/ICEET53442.2021.9659771>
- [22] Non-Fungible Tokens (NFTs) Explained. (n.d.). Amazon Web Services, Inc. Retrieved February 20, 2023, from <https://aws.amazon.com/blockchain/nfts-explained/>
- [23] Hickey, B. (2022). How to put your NFT on another blockchain: Cross-chain NFTs. *Finder.Com*. <https://www.finder.com/cross-chain-nfts>
- [24] Liu, C. G., Bodorik, P., & Jutla, D. (2022). Supporting Long-term Transactions in Smart Contracts. 2022 Fourth International Conference on Blockchain Computing and Applications (BCCA), 11–19. <https://doi.org/10.1109/BCCA55292.2022.9922193>

- [25] HeartBank®. (2018). Smart Contract Design Patterns. HeartBank Studio. Retrieved February 20, 2023, from <https://medium.com/heartbankstudio/smart-contract-design-patterns-8b7ca8b80dfb>
- [26] Opcodes for the EVM. (n.d.). Ethereum.Org. Retrieved March 14, 2023, from <https://ethereum.org>
- [27] Nested transaction. (2019). In Wikipedia. https://en.wikipedia.org/w/index.php?title=Nested_transaction&oldid=931313025
- [28] Blockchain Technology: Layer-1 and Layer-2 Networks. (n.d.). Gemini. Retrieved February 21, 2023, from <https://www.gemini.com/cryptopedia/blockchain-layer-2-network-layer-1-network>
- [29] Non-Fungible Tokens (NFTs) Explained | AWS Blockchain. (n.d.). Amazon Web Services, Inc. Retrieved February 20, 2023, from <https://aws.amazon.com/blockchain/nfts-explained/>
- [30] Non-fungible token. (2023). In Wikipedia. https://en.wikipedia.org/w/index.php?title=Non-fungible_token&oldid=1142225136

HISTORY DATES

Submitted March 2023;